

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>DSA Visualizer | Bidisha Dutta</title>
<link
href="https://fonts.googleapis.com/css2?family=JetBrains+Mono:wght@400;500;700&family=Sy
ne:wght@400;600;800&display=swap" rel="stylesheet">
<style>
:root {
  --bg: #0a0a0f;
  --surface: #12121a;
  --surface2: #1a1a26;
  --border: #2a2a3d;
  --accent: #7c6af7;
  --accent2: #4fd1c5;
  --warn: #f6ad55;
  --danger: #fc8181;
  --success: #68d391;
  --text: #e2e8f0;
  --muted: #718096;
  --mono: 'JetBrains Mono', monospace;
  --sans: 'Syne', sans-serif;
}
* { box-sizing: border-box; margin: 0; padding: 0; }
body { background: var(--bg); color: var(--text); font-family: var(--sans); min-height: 100vh;
overflow-x: hidden; }

/* Background grid */
body::before {
  content: "";
  position: fixed; inset: 0;
  background-image: linear-gradient(var(--border) 1px, transparent 1px), linear-gradient(90deg,
var(--border) 1px, transparent 1px);
  background-size: 40px 40px;
  opacity: 0.3;
  pointer-events: none;
}

header {
  padding: 2rem 3rem 1.5rem;
  border-bottom: 1px solid var(--border);
  display: flex; align-items: center; justify-content: space-between;
```

```

    position: relative;
}
.logo { font-size: 1.4rem; font-weight: 800; letter-spacing: -0.5px; }
.logo span { color: var(--accent); }
.tag { font-family: var(--mono); font-size: 11px; color: var(--muted); background: var(--surface2);
border: 1px solid var(--border); padding: 4px 10px; border-radius: 4px; }

.tabs {
    display: flex; gap: 0; padding: 1.5rem 3rem 0;
    border-bottom: 1px solid var(--border);
}
.tab {
    font-family: var(--mono); font-size: 13px; padding: 10px 24px;
    background: none; border: none; color: var(--muted); cursor: pointer;
    border-bottom: 2px solid transparent; margin-bottom: -1px;
    transition: color 0.2s, border-color 0.2s;
}
.tab:hover { color: var(--text); }
.tab.active { color: var(--accent); border-bottom-color: var(--accent); }

.main { padding: 2rem 3rem; max-width: 1100px; margin: 0 auto; }

.panel { display: none; }
.panel.active { display: block; }

/* Controls */
.controls-row {
    display: flex; gap: 10px; align-items: center; flex-wrap: wrap; margin-bottom: 1.5rem;
}
select {
    font-family: var(--mono); font-size: 12px; background: var(--surface2);
    border: 1px solid var(--border); color: var(--text); padding: 8px 14px;
    border-radius: 6px; cursor: pointer; outline: none;
}
select:hover { border-color: var(--accent); }
button {
    font-family: var(--mono); font-size: 12px; padding: 8px 18px;
    border-radius: 6px; cursor: pointer; border: 1px solid var(--border);
    background: var(--surface2); color: var(--text); transition: all 0.15s;
}
button:hover { border-color: var(--accent); color: var(--accent); }
button.primary { background: var(--accent); color: white; border-color: var(--accent); }
button.primary:hover { opacity: 0.85; }
button.danger-btn { border-color: var(--danger); color: var(--danger); }

```

```

.slider-group { display: flex; align-items: center; gap: 8px; }
.slider-group label { font-family: var(--mono); font-size: 11px; color: var(--muted); }
input[type=range] {
  -webkit-appearance: none; width: 100px; height: 4px;
  background: var(--border); border-radius: 2px; outline: none;
}
input[type=range]::-webkit-slider-thumb {
  -webkit-appearance: none; width: 14px; height: 14px;
  border-radius: 50%; background: var(--accent); cursor: pointer;
}
.slider-val { font-family: var(--mono); font-size: 11px; color: var(--accent); min-width: 28px; }

/* Stats bar */
.stats-bar {
  display: flex; gap: 12px; margin-bottom: 1.5rem; flex-wrap: wrap;
}
.stat {
  background: var(--surface); border: 1px solid var(--border);
  border-radius: 8px; padding: 8px 16px;
  display: flex; flex-direction: column; gap: 2px;
}
.stat-label { font-family: var(--mono); font-size: 10px; color: var(--muted); text-transform:
uppercase; letter-spacing: 0.5px; }
.stat-value { font-family: var(--mono); font-size: 14px; font-weight: 700; color: var(--accent2); }

/* Canvas */
.canvas-wrap {
  background: var(--surface); border: 1px solid var(--border);
  border-radius: 12px; overflow: hidden; margin-bottom: 1rem;
}
canvas { display: block; width: 100%; }

/* Legend */
.legend { display: flex; gap: 16px; flex-wrap: wrap; }
.legend-item { display: flex; align-items: center; gap: 6px; font-family: var(--mono); font-size:
11px; color: var(--muted); }
.legend-dot { width: 10px; height: 10px; border-radius: 50%; flex-shrink: 0; }

/* Status */
.status-line {
  font-family: var(--mono); font-size: 12px; color: var(--muted);
  margin-top: 10px; min-height: 18px;
  padding: 8px 14px; background: var(--surface); border-left: 3px solid var(--accent);
}

```

```

border-radius: 0 6px 6px 0;
}
.status-line.success { border-left-color: var(--success); color: var(--success); }
.status-line.warn { border-left-color: var(--warn); color: var(--warn); }

/* Path tool info */
.path-info { font-family: var(--mono); font-size: 11px; color: var(--muted); margin-bottom: 10px; }
</style>
</head>
<body>

<header>
  <div class="logo">DSA<span>.</span>viz</div>
  <div class="tag">Sorting · Graph · Pathfinding</div>
</header>

<div class="tabs">
  <button class="tab active" onclick="switchTab('sort')">Sorting Algorithms</button>
  <button class="tab" onclick="switchTab('graph')">BFS / DFS</button>
  <button class="tab" onclick="switchTab('path')">Pathfinding</button>
</div>

<div class="main">

  <!-- SORTING -->
  <div id="sort-panel" class="panel active">
    <div class="controls-row">
      <select id="sort-algo" onchange="onAlgoChange()">
        <option value="bubble">Bubble Sort</option>
        <option value="selection">Selection Sort</option>
        <option value="insertion">Insertion Sort</option>
        <option value="merge">Merge Sort</option>
        <option value="quick">Quick Sort</option>
      </select>
      <div class="slider-group">
        <label>SIZE</label>
        <input type="range" id="arr-size" min="8" max="60" value="30" step="1"
oninput="sizeLabel.textContent=this.value;generateArray()">
        <span class="slider-val" id="sizeLabel">30</span>
      </div>
      <div class="slider-group">
        <label>SPEED</label>
        <input type="range" id="sort-speed" min="10" max="300" value="120" step="10">
      </div>
    </div>
  </div>

```

```

        <button onclick="generateArray()">Shuffle</button>
        <button class="primary" onclick="startSort()">Run ▶</button>
        <button class="danger-btn" onclick="stopAll()">Stop ■</button>
    </div>
    <div class="stats-bar">
        <div class="stat"><span class="stat-label">Algorithm</span><span class="stat-value"
id="s-algo">Bubble Sort</span></div>
        <div class="stat"><span class="stat-label">Time Complexity</span><span
class="stat-value" id="s-time">O(n2)</span></div>
        <div class="stat"><span class="stat-label">Space</span><span class="stat-value"
id="s-space">O(1)</span></div>
        <div class="stat"><span class="stat-label">Comparisons</span><span class="stat-value"
id="s-cmp">0</span></div>
        <div class="stat"><span class="stat-label">Swaps</span><span class="stat-value"
id="s-swp">0</span></div>
    </div>
    <div class="canvas-wrap"><canvas id="sort-canvas" height="260"></canvas></div>
    <div class="legend">
        <div class="legend-item"><div class="legend-dot"
style="background:#7c6af7"></div>Unsorted</div>
        <div class="legend-item"><div class="legend-dot"
style="background:#fc8181"></div>Comparing</div>
        <div class="legend-item"><div class="legend-dot"
style="background:#68d391"></div>Sorted</div>
        <div class="legend-item"><div class="legend-dot" style="background:#f6ad55"></div>Pivot /
Min</div>
    </div>
    <div class="status-line" id="sort-status">Click Run to start the visualization</div>
</div>

<!-- GRAPH -->
<div id="graph-panel" class="panel">
    <div class="controls-row">
        <select id="graph-algo" onchange="resetGraph()">
            <option value="bfs">Breadth-First Search (BFS)</option>
            <option value="dfs">Depth-First Search (DFS)</option>
        </select>
        <button onclick="resetGraph()">New Graph</button>
        <button class="primary" onclick="startGraph()">Run ▶</button>
        <button class="danger-btn" onclick="stopAll()">Stop ■</button>
    </div>
    <div class="stats-bar">
        <div class="stat"><span class="stat-label">Data Structure</span><span class="stat-value"
id="g-ds">Queue</span></div>

```

```

    <div class="stat"><span class="stat-label">Time Complexity</span><span
class="stat-value">O(V+E)</span></div>
    <div class="stat"><span class="stat-label">Space</span><span
class="stat-value">O(V)</span></div>
    <div class="stat"><span class="stat-label">Nodes Visited</span><span class="stat-value"
id="g-visited">0</span></div>
    <div class="stat"><span class="stat-label">Order</span><span class="stat-value"
id="g-order">—</span></div>
</div>
<div class="canvas-wrap"><canvas id="graph-canvas" height="320"></canvas></div>
<div class="legend">
    <div class="legend-item"><div class="legend-dot"
style="background:#7c6af7"></div>Unvisited</div>
    <div class="legend-item"><div class="legend-dot" style="background:#fc8181"></div>In
Queue/Stack</div>
    <div class="legend-item"><div class="legend-dot"
style="background:#68d391"></div>Visited</div>
    <div class="legend-item"><div class="legend-dot"
style="background:#f6ad55"></div>Current</div>
</div>
<div class="status-line" id="graph-status">Click Run to start traversal from node A</div>
</div>

```

```

<!-- PATHFINDING -->
<div id="path-panel" class="panel">
    <div class="controls-row">
        <select id="path-tool">
            <option value="wall">Draw Walls</option>
            <option value="start">Move Start</option>
            <option value="end">Move End</option>
        </select>
        <button onclick="resetGrid()">Clear Grid</button>
        <button onclick="randomWalls()">Random Walls</button>
        <button class="primary" onclick="startDijkstra()">Run Dijkstra ▶</button>
        <button class="danger-btn" onclick="stopAll()">Stop ■</button>
    </div>
    <div class="stats-bar">
        <div class="stat"><span class="stat-label">Algorithm</span><span
class="stat-value">Dijkstra's</span></div>
        <div class="stat"><span class="stat-label">Time</span><span
class="stat-value">O((V+E)logV)</span></div>
        <div class="stat"><span class="stat-label">Nodes Visited</span><span class="stat-value"
id="p-visited">0</span></div>
    </div>

```

```

    <div class="stat"><span class="stat-label">Path Length</span><span class="stat-value"
id="p-length">—</span></div>
  </div>
  <div class="path-info">Click/drag to draw walls · Select tool above to move start (green) or
end (red)</div>
  <div class="canvas-wrap"><canvas id="path-canvas" height="300"
style="cursor:crosshair"></canvas></div>
  <div class="legend">
    <div class="legend-item"><div class="legend-dot"
style="background:#1a1a26"></div>Empty</div>
    <div class="legend-item"><div class="legend-dot"
style="background:#2a2a3d"></div>Wall</div>
    <div class="legend-item"><div class="legend-dot"
style="background:#68d391"></div>Start</div>
    <div class="legend-item"><div class="legend-dot"
style="background:#fc8181"></div>End</div>
    <div class="legend-item"><div class="legend-dot"
style="background:#4a3f7a"></div>Visited</div>
    <div class="legend-item"><div class="legend-dot"
style="background:#f6ad55"></div>Shortest Path</div>
  </div>
  <div class="status-line" id="path-status">Draw walls then click Run Dijkstra</div>
</div>

<script>
let animId = null, running = false;

const algoMeta = {
  bubble: { name:'Bubble Sort',   time:'O(n²)',    space:'O(1)' },
  selection: { name:'Selection Sort', time:'O(n²)',    space:'O(1)' },
  insertion: { name:'Insertion Sort', time:'O(n²)',    space:'O(1)' },
  merge: { name:'Merge Sort',    time:'O(n log n)', space:'O(n)' },
  quick: { name:'Quick Sort',    time:'O(n log n)', space:'O(log n)' }
};

function stopAll() { running = false; if(animId) { clearTimeout(animId); animId = null; } }

function switchTab(tab) {
  stopAll();
  document.querySelectorAll('.panel').forEach(p => p.classList.remove('active'));
  document.querySelectorAll('.tab').forEach(t => t.classList.remove('active'));
  document.getElementById(tab+'-panel').classList.add('active');

```

```

const idx = {sort:0,graph:1,path:2}[tab];
document.querySelectorAll('.tab')[idx].classList.add('active');
if(tab==='sort') generateArray();
if(tab==='graph') resetGraph();
if(tab==='path') resetGrid();
}

```

// ——— SORTING —————

```

let arr = [], colors = [];
let cmp = 0, swp = 0;

```

```

function onAlgoChange() {
  const a = document.getElementById('sort-algo').value;
  const m = algoMeta[a];
  document.getElementById('s-algo').textContent = m.name;
  document.getElementById('s-time').textContent = m.time;
  document.getElementById('s-space').textContent = m.space;
}

```

```

function generateArray() {
  stopAll();
  const n = parseInt(document.getElementById('arr-size').value);
  arr = Array.from({length:n}, () => Math.floor(Math.random()*85)+10);
  colors = new Array(n).fill('default');
  cmp = 0; swp = 0;
  document.getElementById('s-cmp').textContent = '0';
  document.getElementById('s-swp').textContent = '0';
  setStatus('sort-status', 'Click Run to start the visualization', '');
  drawSort();
}

```

```

function drawSort() {
  const c = document.getElementById('sort-canvas');
  const ctx = c.getContext('2d');
  c.width = c.parentElement.clientWidth;
  const W = c.width, H = 260, n = arr.length;
  ctx.clearRect(0,0,W,H);
  const bw = Math.max(2, (W - 20) / n - 1);
  const gap = (W - 20) / n;
  const maxV = Math.max(...arr);
  const colMap = { default:'#7c6af7', comparing:'#fc8181', sorted:'#68d391', pivot:'#f6ad55' };
  arr.forEach((v, i) => {
    const h = Math.floor((v / maxV) * (H - 20));
    const x = 10 + i * gap;

```



```

    ctx.fillStyle = colMap[colors[i]] || '#7c6af7';
    ctx.beginPath();
    ctx.roundRect ? ctx.roundRect(x, H-h-4, bw, h, [3,3,0,0]) : ctx.rect(x, H-h-4, bw, h);
    ctx.fill();
  });
}

```

```

function setStatus(id, msg, type='') {
  const el = document.getElementById(id);
  el.textContent = msg;
  el.className = 'status-line' + (type ? ' '+type : '');
}

```

```

function sleep(ms) { return new Promise(r => { animId = setTimeout(r, ms); }); }
function spd() { return Math.max(5, 320 -
  parseInt(document.getElementById('sort-speed').value)); }
function upd() {
  drawSort();
  document.getElementById('s-cmp').textContent = cmp;
  document.getElementById('s-swp').textContent = swp;
}

```

```

async function startSort() {
  stopAll(); running = true;
  const a = arr.slice();
  colors = new Array(a.length).fill('default');
  cmp = 0; swp = 0;
  const algo = document.getElementById('sort-algo').value;
  setStatus('sort-status', `Running ${algoMeta[algo].name}...`);

```

```

  if(algo === 'bubble') {
    for(let i = 0; i < a.length && running; i++) {
      for(let j = 0; j < a.length-i-1 && running; j++) {
        colors[j]='comparing'; colors[j+1]='comparing'; arr=a.slice(); upd(); await sleep(spd());
        cmp++;
        if(a[j] > a[j+1]) { [a[j],a[j+1]]=[a[j+1],a[j]]; swp++; }
        colors[j]='default'; colors[j+1]='default';
      }
      colors[a.length-1-i]='sorted'; arr=a.slice(); upd();
    }
  } else if(algo === 'selection') {
    for(let i = 0; i < a.length && running; i++) {
      let mi = i; colors[i]='pivot';
      for(let j = i+1; j < a.length && running; j++) {

```

```

        colors[j]='comparing'; arr=a.slice(); upd(); await sleep(spд()); cmp++;
        if(a[j] < a[mi]) { if(mi!==i) colors[mi]='default'; mi=j; colors[j]='pivot'; }
        else colors[j]='default';
    }
    if(mi!==i) { [a[i],a[mi]]=[a[mi],a[i]]; swp++; }
    colors[i]='sorted'; if(mi!==i) colors[mi]='default'; arr=a.slice(); upd();
}
} else if(algo === 'insertion') {
    colors[0]='sorted';
    for(let i = 1; i < a.length && running; i++) {
        let key=a[i], j=i-1; colors[i]='comparing'; arr=a.slice(); upd(); await sleep(spд());
        while(j >= 0 && a[j] > key && running) {
            cmp++; a[j+1]=a[j]; colors[j]='comparing'; arr=a.slice(); upd(); await sleep(spд());
            colors[j]='sorted'; j--;
        }
        a[j+1]=key; swp++; colors[i]='sorted'; arr=a.slice(); upd();
    }
} else if(algo === 'merge') {
    await mergeSort(a, 0, a.length-1);
} else if(algo === 'quick') {
    await quickSort(a, 0, a.length-1);
}

if(running) {
    colors = new Array(a.length).fill('sorted'); arr=a.slice(); upd();
    setStatus('sort-status', `Done! ${cmp} comparisons, ${swp} swaps`, 'success');
}
}

async function mergeSort(a, l, r) {
    if(l >= r || !running) return;
    const m = Math.floor((l+r)/2);
    await mergeSort(a, l, m); await mergeSort(a, m+1, r);
    const L=a.slice(l,m+1), R=a.slice(m+1,r+1);
    let i=0,j=0,k=l;
    while(i<L.length && j<R.length && running) {
        cmp++; colors[k]='comparing'; arr=a.slice(); upd(); await sleep(spд());
        a[k++] = L[i]<=R[j] ? L[i++] : (swp++, R[j++]);
        colors[k-1]='sorted';
    }
    while(i<L.length) a[k++]=L[i++];
    while(j<R.length) a[k++]=R[j++];
    arr=a.slice(); upd();
}
}

```

```

async function quickSort(a, lo, hi) {
  if(lo >= hi || !running) return;
  let pivot=a[hi], i=lo-1; colors[hi]='pivot';
  for(let j=lo; j<hi && running; j++) {
    cmp++; colors[j]='comparing'; arr=a.slice(); upd(); await sleep(spd());
    if(a[j]<=pivot) { i++; [a[i],a[j]]=a[j],a[i]; swp++; }
    colors[j]='default';
  }
  [a[i+1],a[hi]]=a[hi],a[i+1]; colors[hi]='default'; colors[i+1]='sorted';
  arr=a.slice(); upd();
  await quickSort(a, lo, i); await quickSort(a, i+2, hi);
}

```

```

// ——— GRAPH —————
let gNodes=[], gEdges=[], gColors=[];

function resetGraph() {
  stopAll();
  const algo = document.getElementById('graph-algo').value;
  document.getElementById('g-ds').textContent = algo==='bfs' ? 'Queue (FIFO)' : 'Stack (LIFO)';
  document.getElementById('g-visited').textContent = '0';
  document.getElementById('g-order').textContent = '—';
  setStatus('graph-status','Click Run to start traversal from node A');
  gNodes=[]; gEdges=[];
  const c = document.getElementById('graph-canvas');
  c.width = c.parentElement.clientWidth;
  const W=c.width, H=320;
  const pos = [
    [W*.5,50],[W*.2,130],[W*.8,130],[W*.05,240],[W*.35,240],[W*.65,240],[W*.95,240],[W*.5,240]
  ];
  pos.forEach((p,i)=>gNodes.push({x:p[0],y:p[1],label:String.fromCharCode(65+i)}));
  [[0,1],[0,2],[1,3],[1,4],[2,5],[2,6],[4,7],[5,7]].forEach(e=>gEdges.push(e));
  gColors = new Array(gNodes.length).fill('default');
  drawGraph();
}

```

```

function drawGraph() {
  const c = document.getElementById('graph-canvas');
  const ctx = c.getContext('2d');
  ctx.clearRect(0,0,c.width,c.height);
  const cm = { default:'#7c6af7', frontier:'#fc8181', visited:'#68d391', current:'#f6ad55' };
  gEdges.forEach(([a,b])=>{
    ctx.beginPath(); ctx.moveTo(gNodes[a].x,gNodes[a].y); ctx.lineTo(gNodes[b].x,gNodes[b].y);

```

```

    ctx.strokeStyle='#2a2a3d'; ctx.lineWidth=2; ctx.stroke();
  });
  gNodes.forEach((n,i)=>{
    ctx.beginPath(); ctx.arc(n.x,n.y,22,0,Math.PI*2);
    ctx.fillStyle=cm[gColors[i]]|#7c6af7'; ctx.fill();
    ctx.strokeStyle='#0a0a0f'; ctx.lineWidth=3; ctx.stroke();
    ctx.fillStyle='#0a0a0f'; ctx.font='bold 13px Syne,sans-serif';
    ctx.textAlign='center'; ctx.textBaseline='middle';
    ctx.fillText(n.label,n.x,n.y);
  });
}

async function startGraph() {
  stopAll(); running=true;
  gColors=new Array(gNodes.length).fill('default');
  const algo=document.getElementById('graph-algo').value;
  const adj={}; gNodes.forEach(n=>adj[n.id||gNodes.indexOf(n)]=[]);
  gEdges.forEach(([a,b])=>{adj[a].push(b);adj[b].push(a);});
  let visitedCount=0, orderArr=[];

  if(algo==='bfs') {
    const queue=[0], visited=new Set([0]); gColors[0]='frontier';
    while(queue.length && running) {
      const cur=queue.shift(); gColors[cur]='current'; drawGraph(); await sleep(700);
      visitedCount++; orderArr.push(gNodes[cur].label);
      document.getElementById('g-visited').textContent=visitedCount;
      document.getElementById('g-order').textContent=orderArr.join('→');
      setStatus('graph-status',`Visiting ${gNodes[cur].label} | Queue:
[${queue.map(q=>gNodes[q].label).join(', ')}|'empty']`);
      gColors[cur]='visited';
      for(const nb of adj[cur]) {
        if(!visited.has(nb)){visited.add(nb);gColors[nb]='frontier';queue.push(nb);} }
      drawGraph(); await sleep(300);
    }
  } else {
    const stack=[0], visited=new Set();
    while(stack.length && running) {
      const cur=stack.pop();
      if(visited.has(cur)) continue;
      visited.add(cur); gColors[cur]='current'; drawGraph(); await sleep(700);
      visitedCount++; orderArr.push(gNodes[cur].label);
      document.getElementById('g-visited').textContent=visitedCount;
      document.getElementById('g-order').textContent=orderArr.join('→');
    }
  }
}

```

```

    setStatus('graph-status', `Visiting ${gNodes[cur].label} | Stack:
[${stack.map(q=>gNodes[q].label).join(', ')}||'empty']`);
    gColors[cur]='visited';
    for(const nb of adj[cur]){if(!visited.has(nb)){gColors[nb]='frontier';stack.push(nb);}}
    drawGraph(); await sleep(300);
  }
}
if(running) setStatus('graph-status', `Complete! Traversal order: ${orderArr.join(' → ')}`,
'success');
}

```

// ——— PATHFINDING ———

```

const ROWS=15, COLS=28;
let grid=[], startR=7, startC=2, endR=7, endC=25, mouseDown=false;

```

```

function resetGrid() {
  stopAll();
  grid=Array.from({length:ROWS},()=>new Array(COLS).fill(0));
  document.getElementById('p-visited').textContent='0';
  document.getElementById('p-length').textContent='—';
  setStatus('path-status','Draw walls then click Run Dijkstra');
  drawGrid();
}

```

```

function randomWalls() {
  resetGrid();
  for(let r=0;r<ROWS;r++) for(let c=0;c<COLS;c++) {
    if((r===startR&&c===startC)||((r===endR&&c===endC))) continue;
    if(Math.random()<0.28) grid[r][c]=1;
  }
  drawGrid();
}

```

```

function cellSize() {
  const c=document.getElementById('path-canvas');
  return { cw: c.width/COLS, ch: 300/ROWS };
}

```

```

function drawGrid() {
  const c=document.getElementById('path-canvas');
  const ctx=c.getContext('2d');
  c.width=c.parentElement.clientWidth;
  const W=c.width, H=300, cw=W/COLS, ch=H/ROWS;
  ctx.clearRect(0,0,W,H);

```

```

const cm={0:'#12121a',1:'#2a2a3d',4:'#4a3f7a',5:'#f6ad55'};
for(let r=0;r<ROWS;r++) for(let cl=0;cl<COLS;cl++) {
  const isS=r===startR&&cl===startC, isE=r===endR&&cl===endC;
  ctx.fillStyle=isS?'#68d391':isE?'#fc8181':cm[grid[r][cl]]||'#12121a';
  ctx.fillRect(cl*cw+1,r*ch+1,cw-2,ch-2);
  if(isS||isE) {
    ctx.fillStyle='#0a0a0f'; ctx.font=`bold ${Math.floor(ch*0.55)}px Syne,sans-serif`;
    ctx.textAlign='center'; ctx.textBaseline='middle';
    ctx.fillText(isS?'S':'E', cl*cw+cw/2, r*ch+ch/2);
  }
}
ctx.strokeStyle='#0a0a0f'; ctx.lineWidth=0.5;
for(let r=0;r<=ROWS;r++){ctx.beginPath();ctx.moveTo(0,r*ch);ctx.lineTo(W,r*ch);ctx.stroke();}
for(let
cl=0;cl<=COLS;cl++){ctx.beginPath();ctx.moveTo(cl*cw,0);ctx.lineTo(cl*cw,H);ctx.stroke();}
}

function getCell(e) {
  const c=document.getElementById('path-canvas');
  const rect=c.getBoundingClientRect();
  const x=e.clientX-rect.left, y=e.clientY-rect.top;
  const cw=c.width/COLS, ch=300/ROWS;
  return [Math.floor(y/ch), Math.floor(x/cw)];
}

function handlePathClick(e) {
  const [r,cl]=getCell(e);
  if(r<0||r>=ROWS||cl<0||cl>=COLS) return;
  const tool=document.getElementById('path-tool').value;
  if(tool==='wall') { if(!((r===startR&&cl===startC)||((r===endR&&cl===endC)))
grid[r][cl]=grid[r][cl]?0:1; }
  else if(tool==='start') { startR=r; startC=cl; }
  else { endR=r; endC=cl; }
  drawGrid();
}

function handlePathDrag(e) {
  if(!mouseDown) return;
  const [r,cl]=getCell(e);
  if(r<0||r>=ROWS||cl<0||cl>=COLS) return;
  const tool=document.getElementById('path-tool').value;
  if(tool==='wall'&&!((r===startR&&cl===startC)||((r===endR&&cl===endC))) { grid[r][cl]=1;
drawGrid(); }
}

```

```

const pc=document.getElementById('path-canvas');
pc.addEventListener('mousedown',e=>{mouseDown=true;handlePathClick(e);});
pc.addEventListener('mousemove',handlePathDrag);
pc.addEventListener('mouseup',()=>mouseDown=false);
pc.addEventListener('mouseleave',()=>mouseDown=false);

async function startDijkstra() {
  stopAll(); running=true;
  const dist=Array.from({length:ROWS},()=>new Array(COLS).fill(Infinity));
  const prev=Array.from({length:ROWS},()=>new Array(COLS).fill(null));
  const visited=Array.from({length:ROWS},()=>new Array(COLS).fill(false));
  dist[startR][startC]=0;
  const pq=[[0,startR,startC]];
  let visitedCount=0;
  const dirs=[[0,1],[1,0],[0,-1],[-1,0]];
  setStatus('path-status','Searching...');

  while(pq.length && running) {
    pq.sort((a,b)=>a[0]-b[0]);
    const [d,r,cl]=pq.shift();
    if(visited[r][cl]) continue;
    visited[r][cl]=true; visitedCount++;
    document.getElementById('p-visited').textContent=visitedCount;
    if(!((r===startR&&cl===startC)||(r===endR&&cl===endC))) { grid[r][cl]=4; drawGrid(); await
sleep(18); }
    if(r===endR&&cl===endC) break;
    for(const [dr,dc] of dirs) {
      const nr=r+dr,nc=cl+dc;
      if(nr<0||nr>=ROWS||nc<0||nc>=COLS||visited[nr][nc]||grid[nr][nc]===1) continue;
      if(d+1<dist[nr][nc]) { dist[nr][nc]=d+1; prev[nr][nc]=[r,cl]; pq.push([d+1,nr,nc]); }
    }
  }
  if(!running) return;
  if(dist[endR][endC]===Infinity) { setStatus('path-status','No path found — try removing some
walls','warn'); return; }
  let cur=[endR,endC], pathLen=0;
  while(cur&&!(cur[0]===startR&&cur[1]===startC)) {
    if(!(cur[0]===endR&&cur[1]===endC)) { grid[cur[0]][cur[1]]=5; drawGrid(); await sleep(40); }
    cur=prev[cur[0]][cur[1]]; pathLen++;
  }
  document.getElementById('p-length').textContent=pathLen;
  setStatus('path-status','Shortest path found: ${pathLen} steps through ${visitedCount} nodes
visited','success');
}

```

```
// init  
generateArray();  
</script>  
</body>  
</html>
```